

Prediction of 5-minute Bitcoin contracts on Polymarket

Marco Tchernychev, Aynaz Namik, Leonardo Molina

Class: High Frequency Trading Technologies

Professor Matthew Belcher

Introduction and Context

In the present day, prediction markets are rising in popularity, specifically Kalshi and Polymarket. In these markets, one can predict the outcome of events like: ‘Who will win the NBA championship?’ or ‘What will the Federal Reserve System decide in June?’. For this project, the main prediction market studied was Polymarket, a decentralized prediction market. Specifically, we intended on focusing on the 5 and 15-minute bitcoin contracts that this platform provides.

Generally speaking, if one were looking at the 5 and 15-minute contracts and decided to buy shares, then this means that the buyer is betting on whether the price of that specific bitcoin will go above or below the value it started at. Traders buy shares priced between \$0 and \$1, which indicates the probability of the outcome of the event. For example, assume that the 5-minute Bitcoin contracts are trading at \$0.70; this indicates the general public assigns a 70% probability that the Bitcoin spot price will be greater than it was at the start of the interval. At the end of the time interval, Polymarket closes the contract and checks the Chainlink oracle to see if the price of Bitcoin went up or down in that time interval. Once the contract is resolved, Polymarket pays out \$1 to correct predictions and \$0 to incorrect predictions. Referencing that specific example, whomever bought the \$0.70 contract will receive \$1.00, indicating a \$0.30 profit.

The focus of this project is accurately predicting Bitcoin prices at the end of each 5-minute interval. Once a signal of the direction of the Bitcoin price is acquired, the goal would be to trade on that information in the hopes of making a profit. The process, results, challenges, and outcomes are outlined in the following sections.

Challenges Overcome

In the construction of the project, there were technical challenges that needed to be overcome. The first challenge was that Polymarket is blocked in the United States and is illegal to trade. This meant that any connection to Polymarket APIs to collect data was blocked upon

response. Since the project relies on Polymarket data to align our bitcoin price predictions, the collection of this data was vital for progress in the project. Upon researching, Mullvad VPN was chosen to safely send requests to Polymarket APIs (Gamma, Clob, and Data) and to be able to collect market data.

Now, we had the data collection script moved to a server based in NYC. Unfortunately, upon activation, Mullvad would reroute all network traffic through the VPN tunnel, including SSH commands. This resulted in the server constantly being reset, so we then had to move the project to a Docker container that exposes a proxy port for routing network traffic. This resolved the challenge of receiving market data from Polymarket for 5 and 15-minute contracts. Bitcoin data collection was performed from the Coinbase Advanced trade websocket and had no complications.

Once the data was collected, analysis and feature engineering were conducted to allow for the creation of signals that could be traded. The next challenge to overcome was in determining which features would have the highest predictive power and models that would perform the best. There were many trade-offs with this, including the models potentially being slow based on the amount of inference time needed to make a prediction. Additionally, the features to use were unclear at the time of feature engineering, which resulted in experimentation for finding our features to generate an accuracy above 50%. This threshold was chosen because a random guess of the Bitcoin's price is 50%. If the model is able to accurately predict the Bitcoin price with a confidence greater than 50%, the expected value of return would be positive over a period of time.

Design Pivots

With the initial design of our project, we had intended to build a data collection system for both the 5-minute and 15-minute Bitcoin contracts. From there, we hoped to study how the two contracts are priced relative to one another and to the Bitcoin spot price. From there, we wanted to develop a statistical arbitrage strategy that would exploit mispricings between the two contracts. Then, finally, as previously mentioned, we wanted to incorporate sentiment analysis from financial news headlines to generate additional trading signals for Bitcoin and its prediction contracts. This would have been accomplished with natural language processing models, like FinBERT (which is a model that specifically analyzes the sentiment of financial texts).

However, in attempting to collect data for the 15-minute contracts, we couldn't collect it in a manner that was consistent; the number of 15-minute contracts was not nearly enough when we were running it, and we could not find a correlation between the 2 contracts. The markets were efficient, and thus we did not see an opportunity to perform an arbitrage. So we decided to pivot and start focusing solely on the 5-minute contracts, specifically, if the output of a 5 minute contracts polymarket prediction could predict the next contract.

Additionally, we also attempted to have a model that predicted the outcome of the current contract based on the first 3 minutes of the total 5 minute contract. Due to insufficient amounts of data, we could not test these models on specific trading algorithms, but these 3

minute models have a higher accuracy than the models that predict the next contract based on the current contract.

Architecture

Feature	Description
volatility	Standard deviation of price within each 5-minute bucket
buy_volume	Total size of BUY trades per bucket
sell_volume	Total size of SELL trades per bucket
order_flow_imbalance	buy_volume - sell_volume
total_volume	buy_volume + sell_volume
momentum	Change in last price between consecutive buckets
vol_adj_momentum	Momentum divided by volatility
prev_up_down	Whether the previous bucket closed up (1) or down (0)
rsi	14-period relative strength index
rsi_centered	RSI - 50, centered around 0 for linear models
ma_10_dist	Normalized distance from the 10-period moving average
macd	MACD line (12-period exponential moving average (EMA) - 26-period EMA)
bb_dist	Bollinger Band distance (price's distance from 20-period mean in standard deviations)
mom_zscore	Momentum z-score over a 20-period rolling window
vol_change	Percent change in volatility
ofi_scaled	Order flow imbalance normalized by total volume
mom_lag_1	Momentum lag from 1 bucket ago
mom_lag_2	Momentum lag for 2 buckets ago

Table 1: Feature descriptions

The target variable was **next_up_down**: whether the next 5-min bucket closes higher than it opened.

When testing, each model tested every possible combination of features, then outputted the combination that provided the highest accuracy for feature selection using the training data. We had an 80-20 split for training and testing data. Here are the accuracy results from the validation for these features:

- **Logistic Regression** (9 features with a 52.10% accuracy):
buy_volume, sell_volume, total_volume, ma_10_dist, macd, bb_dist, ofi_scaled, mom_lag_1, mom_lag_2
- **XGBoost** (9 features with a 53.03% accuracy):
ofi_scaled, volatility, bb_dist, rsi, vol_change, macd, ma_10_dist, total_volume, mom_lag_2
- **Random Forest** (18 features with a 54.43% accuracy):
volatility, buy_volume, sell_volume, order_flow_imbalance, total_volume, momentum,

vol_adj_momentum, prev_up_down, rsi, ma_10_dist, rsi_centered, mom_zscore, macd, bb_dist, vol_change, ofi_scaled, mom_lag_1, mom_lag_2

- **SVM** (5 features with a 52.46% accuracy):
total_volume, prev_up_down, macd, ofi_scaled, mom_lag_1

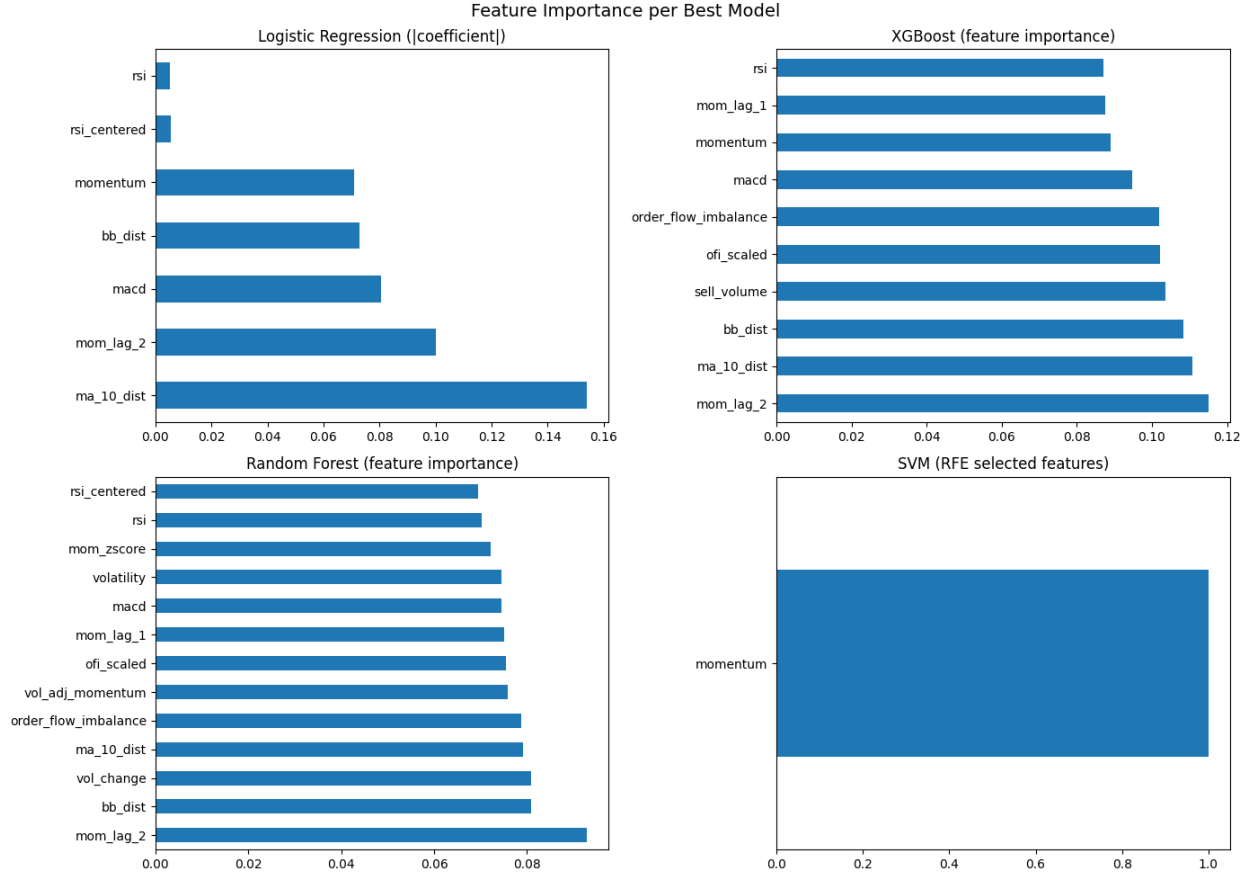


Figure 1: Best Accuracy per Model after Feature Selection

Now, we will provide a brief explanation of how these models work. Please note that the following descriptions are oversimplifications of how complex these models truly are and can be. This section's intentions are simply to give a background understanding:

Logistic Regression is a classifier, not a regressor. It draws a line attempting to separate the 'up' buckets from the 'down' buckets. It then converts the new data points' distance from that line into a probability between 0 and 1. This is the simplest model we used and gives a baseline: if the more complex models can't outperform this, then its features probably do not have much signal.

XGBoost builds a sequence of small decision trees, and each tries to focus on correcting the mistake that the previous tree made. This model is good for making decisions based on context, as a sequence of trees assists in cultivating the best decision for the given context. The trees build off of one another.

Random Forest also uses decision trees, but it builds these in parallel. Meaning that they do not correlate with one another. They are all trained on a random subset of the features and data. It then averages together the accuracy that all the trees produced. The randomness of this model is the focus, with the hopes of the disagreement between these trees preventing overfitting of data, which individual trees tend to do. Overfitting is a phenomenon where the model is too trained on specific data, and becomes skilled in predicting because it is too used to that specific data it has seen and not actually because it has pattern recognition. This is always why, when training models, the full 100% of data is not used to train it. It is usually split between training and testing to see if it is able to give accurate results.

Support Vector Machine (SVM) finds the boundary that separates two classes with the largest possible margin. It focuses on the ‘hardest to classify’ points (the support vectors) right near the boundary. Specifically, SVM selects the features that together define the clearest decision boundary between ‘up’ and ‘down’ outcomes. It also captures nonlinear patterns that logistic regression would miss since we used a radial basis function (RBF) kernel, which bends boundaries into curves. So, its feature chart appears uniform because it does not provide a ranking, rather, it identifies which combination best separates the two classes.

Observations

The following are bullet points of noteworthy observations found in assessing our models, which overall assisted in us learning the nature of these models and how it worked with our data better:

- `macd` and `ofi_scaled` showed up in all four models, making them the most consistently ‘useful’ signals.
- `mom_zscore` and `rsi_centered` only made it into Random Forest, and Random Forest only “used” them because it took the whole set. Random forest, by nature, usually has a larger pool of features it uses, as it, by nature, is a bunch of random trees that use a subset of the total amount of features given to the model. This means that its individual models never see all of the data at once, meaning that it usually needs a wider range of data to give the same accuracy of results.
- RF and SVM agree on 70.00% of predictions.
- XGBoost: volatility is a higher feature here, which indicates how chaotic the market is, which helps, as the nature of the model can switch its logic based on the environment (since it uses decision trees). So knowing its environment helps steer it properly.
- Each contract outcome is independent from previous contract outcome — prevalent in the fact that `prev_up_down` is barely present in this and only in this model.
- `mom_lag_1` is probably the highest for logistic regression because it is a linear model, which means that the most recent thing is usually most influential for the next move (since what we are predicting is very shortly after it).

Results

Here, we explore the results of the remaining 20% of our data split to see how the models react on unseen data. In assessing the results of our models, logistic regression seems to be the best performing, by a minute amount. It performs with a 51.7% accuracy and a p value of .16 (which indicates that this result has a 16% chance of being pure ‘luck’). Close behind is XGBoost, whose performance measures to 51.6% with a p value of .17. These results being so close to logistic regression, despite its nature of a model being more complex, raises concerns. If XGBoost, which captures the relationships of nonlinear values, is not much different than logistic regression, there may not be a general predictive nature. It is also important to note that its p value is slightly worse than linear regression. Random forest has a 49.1% accuracy and p value of .72, heavily leaning towards the idea that its results are simply chance. All of these models did not have a statistically significant edge.

While our SVM was tested for accuracy on the validation set during feature selection, it was excluded from the final probability-scaled evaluation because it does not produce a probability output natively.

Below, we have figures 2, 3, and 4 observing probability-scaled graphs of bankroll simulations from our 3 main models described above. Both logistic regression and XGBoost have a volatile nature, which is why the p value indicates that it could be luck that the window happened to end when it was about 40% above the borderline. For random forest, it ends about 60% below the line, strongly indicating that there is little to no signal.

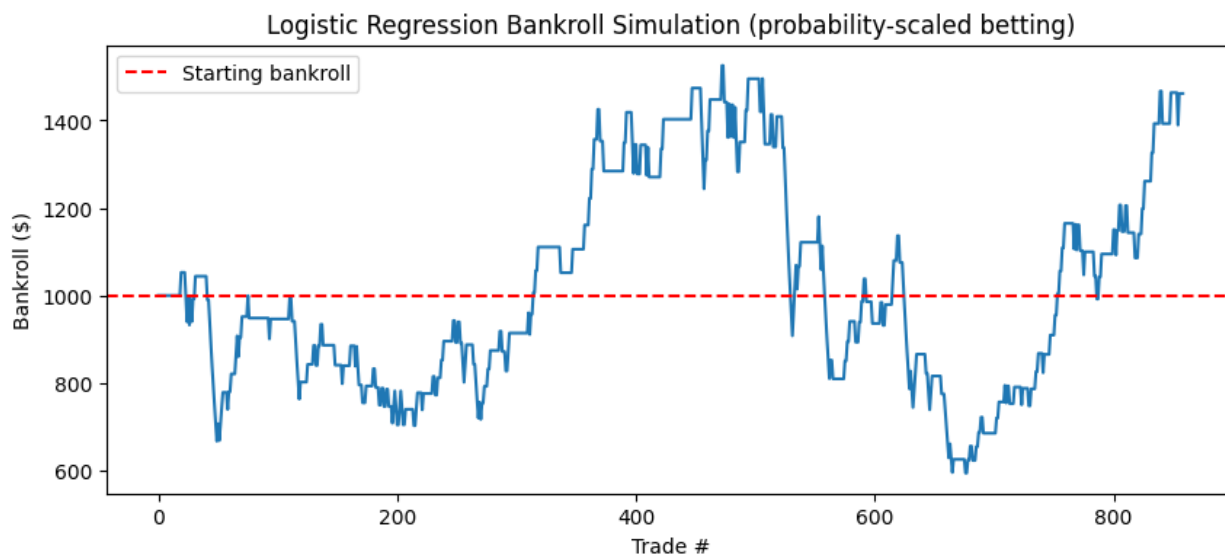


Figure 2: Logistic Regression Bankroll Simulation (probability-scaled betting)

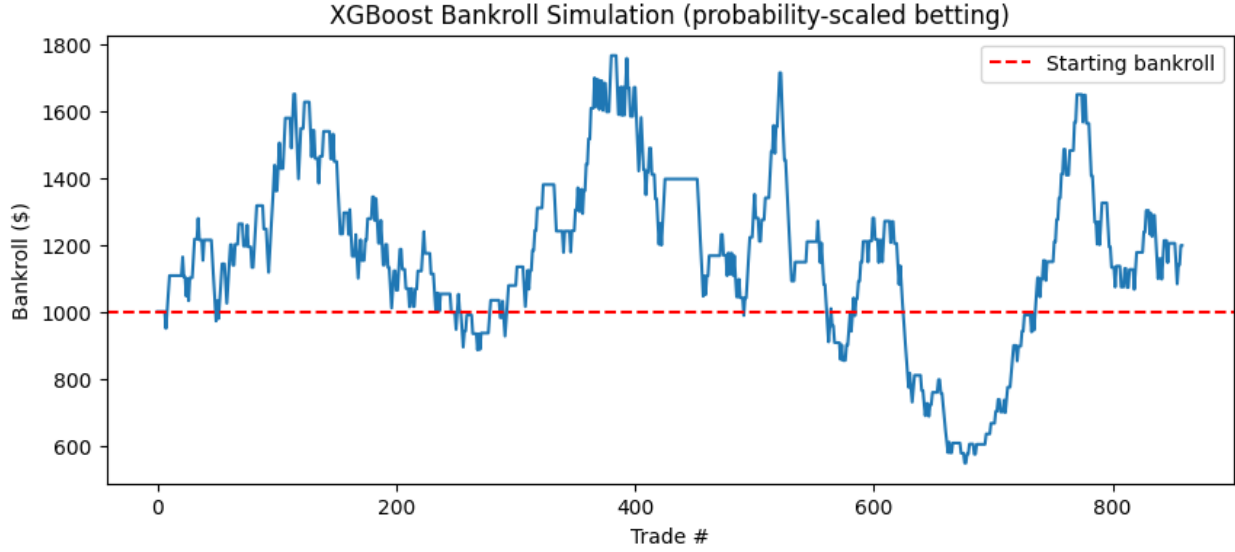


Figure 3: XGBoost Bankroll Simulation (probability-scaled betting)

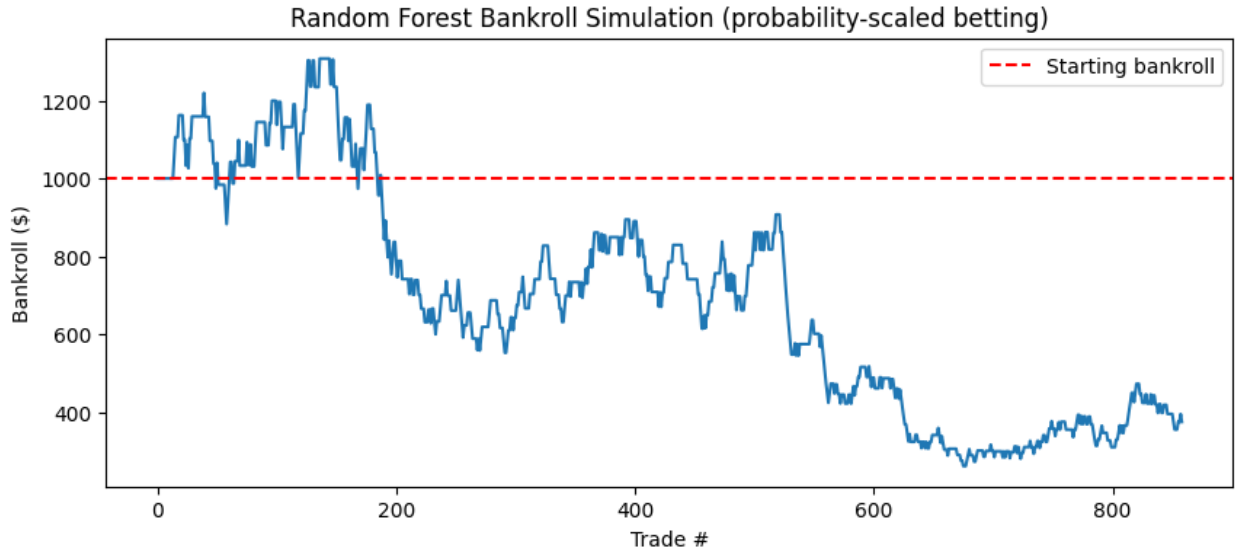


Figure 4: Random Forest Bankroll Simulation (probability-scaled betting)

Future Work & Limitations

Due to PolyMarket having a geographic restriction on the United States, the strategy was not tested live due to legal reasons. The strategy was only tested on data collected over a period of a block of data spanning two weeks. This is not a long enough period of time to tell whether the strategy would actually work in real market data. Thus, for future iterations of

this project, we would like to test our trading strategy on real market data when there is no geographic restriction.

Additionally, we would also like to explore an algorithm that is shaped around the probabilistic outcomes of these models, if it is possible for us to extract a consistent signal in the market's predictive nature. This would be to assess whether or not a real profit can be made in combining these variables with trading software.

Learning Experiences

In the completion of this project, several things were learned, mainly machine learning, data collection, pipelines, and feature engineering. The prediction models used are not generative models like neural networks or GANs. Instead, these models are all used in machine learning. Linear regression was a popular model and thus something we had all been exposed to, whereas XGBoost was not something we had all seen before. This required learning about each individual model and its limitations through trial and error and personal research, in contrast to prior classroom instruction. This aspect was what overlapped greatly with our feature engineering, which was vital to curate features with a high predictive power to nurture an accuracy above 50%. To architect a framework with the proper features, we had to combine our chosen features with our understanding of the models to ensure it properly made sense for their training.

Also, while the data collection was originally thought to be the easiest, it resulted in juggling through loopholes using WireGuard and Docker. Specifically, trying to understand what needed to be done to bypass our IP address being blocked from Polymarket. The project was able to use an existing repository that fleshed out the implementation of a Docker container exposing a port that could be used as a proxy to send our network traffic to Polymarket for data collection. Before finding this repo, we tried to implement this from scratch, overall encouraging a lot of growth in understanding how docker works.

Finally, in this project, the goal was an end-to-end pipeline or trading system. The pipeline followed as such: live data collection, followed by feature engineering, followed by analysis and prediction, and finally placing the trade. This gave us an understanding and practice in building a tradable strategy and tuning it through several iterations.